

Funktionsweise Großer Sprachmodelle (LLMs)

KI-Zyklen bis hin zu modernen LLMs

1950 : Turing-Test als Referenzidee

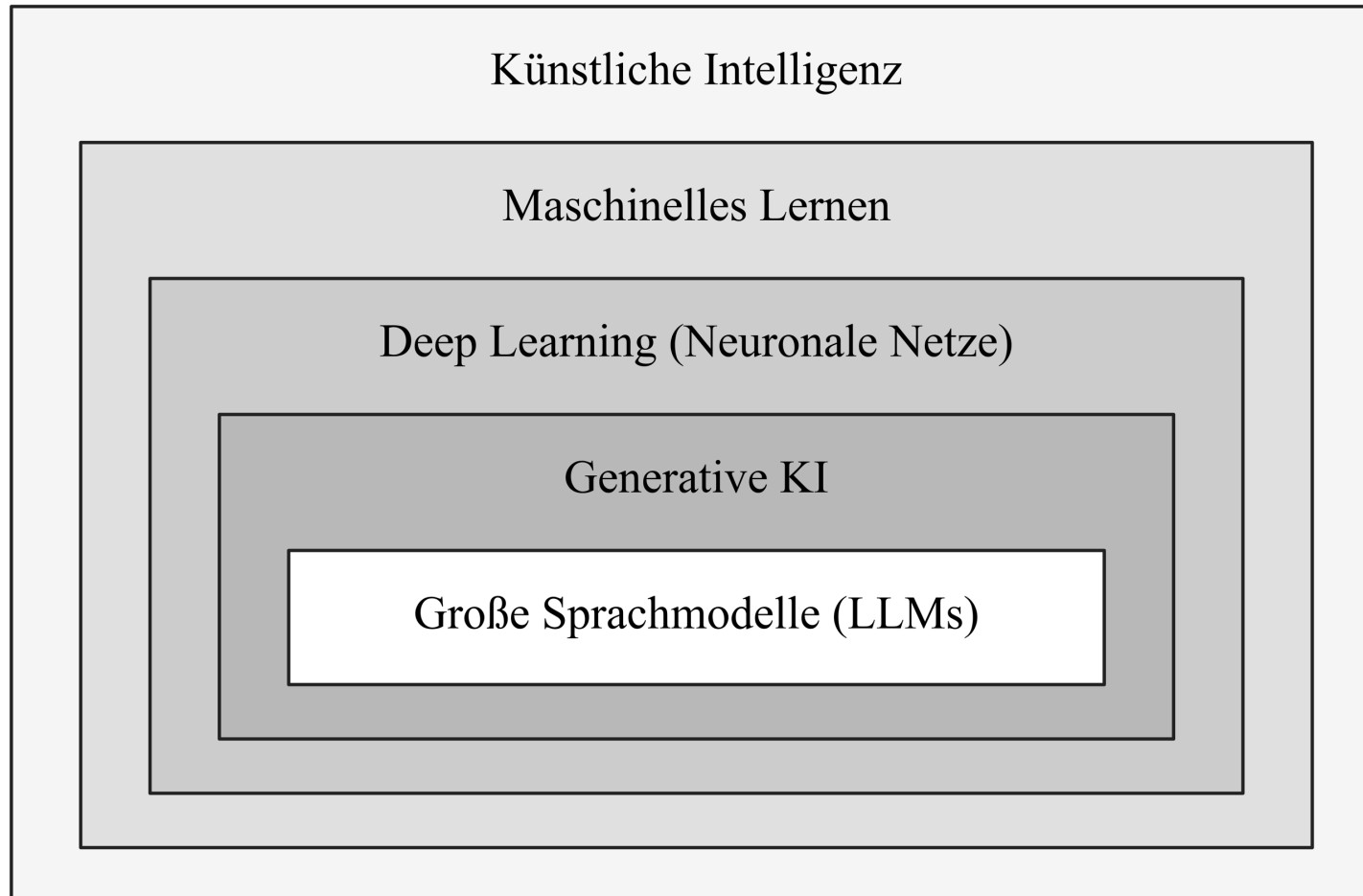
1970 : Expertensysteme (Regeln/Wissen)

2010 : Deep Learning (Skalierung)

2017 : Transformer (Attention)

2020 : Generative Modelle im Mainstream

Begriffe und Systematik



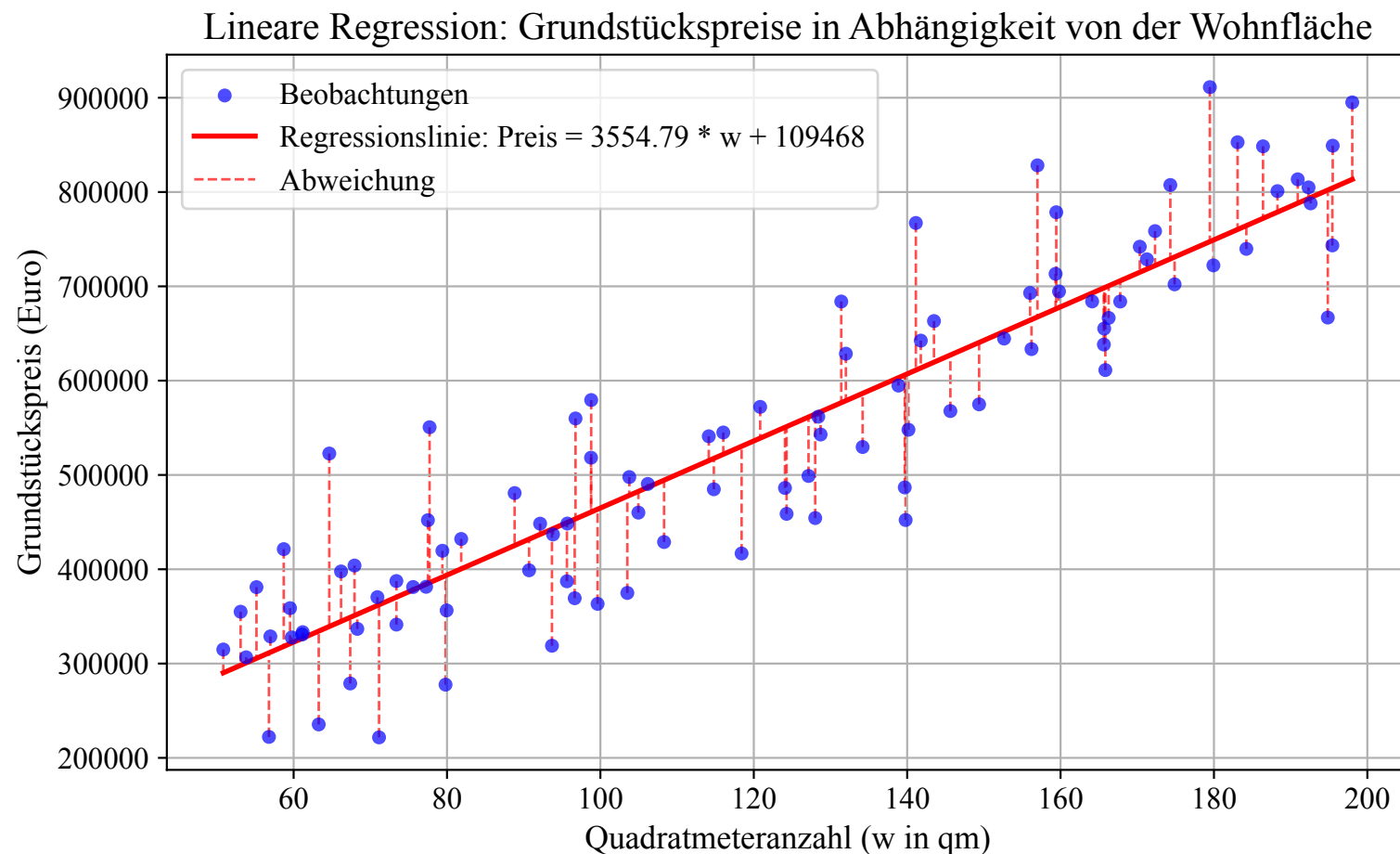
Sonstige
Generative KI—Modelle:

- Bild-Generierung
- Video-Generierung
- Audio-Generierung

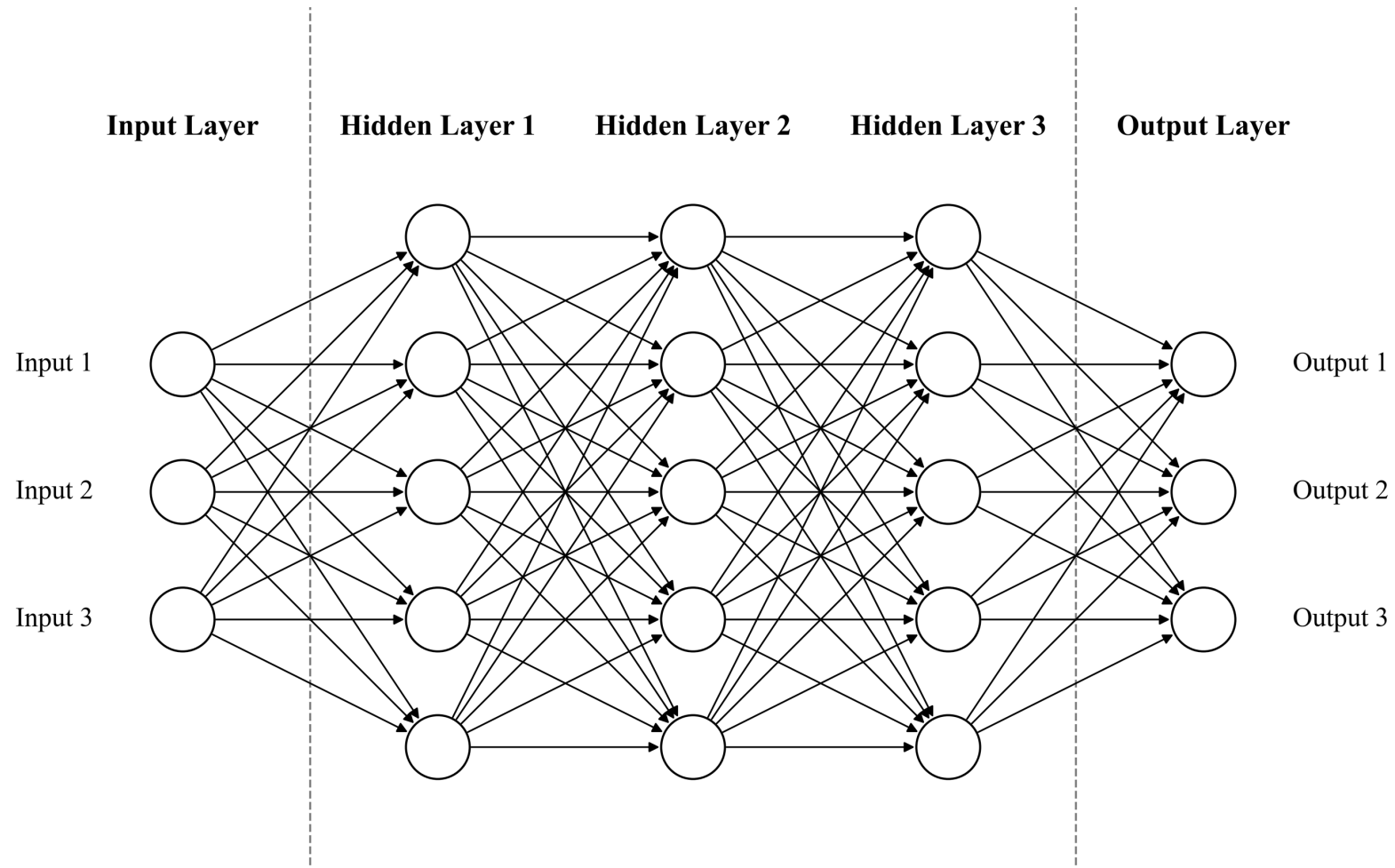
Maschinelles Lernen

- Entwicklung eines Systems, das für Aufgaben eingesetzt werden kann, **ohne es explizit darauf programmieren zu müssen.**
- Gegensatz zur deterministischen Programmierung
- Aufgabe des Trainingsalgorithmus: Die Abweichung von geschätzten Daten zu den tatsächlichen Daten möglichst gering zu halten

Maschinelles Lernen: Beispiel Lineare Regression



Deep Learning: Neuronale Netze



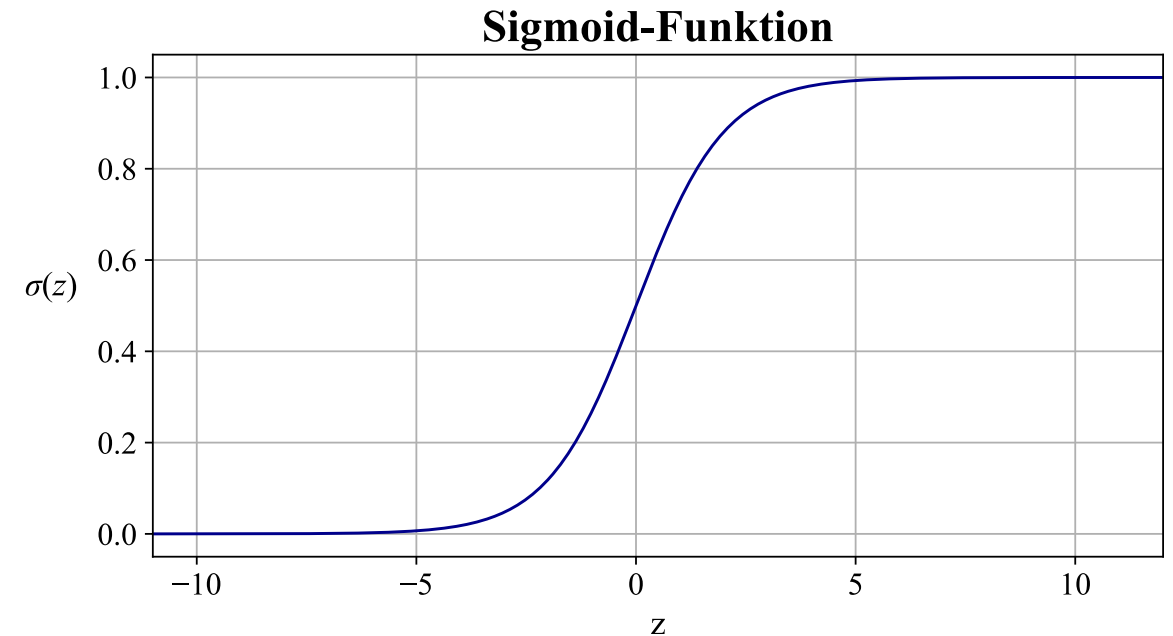
Deep Learning: Neuronale Netze

- „Wie“ der Weitergabe: Lineare Funktion

$$z = w_l h_{l-1} + b_l$$

- „Ob“ der Weitergabe: (nichtlineare) Aktivierungsfunktion

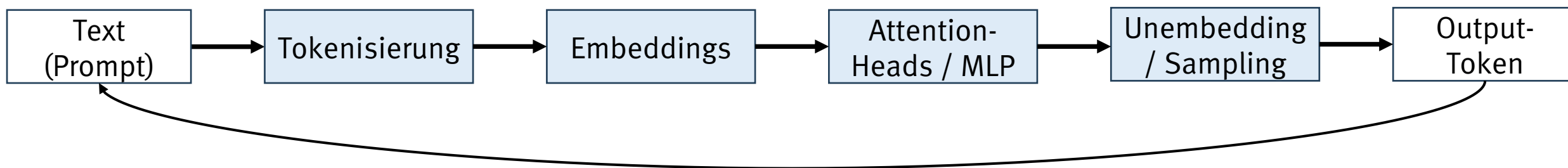
$$h_l = \sigma(z) = \frac{1}{1+e^{-z}}$$



Große Sprachmodelle (LLMs)

- **Sprachmodell:** Jedes auf Basis maschinellen Lernen operierendes Modell, das Wahrscheinlichkeiten für der Eingabesequenz folgende Wörter ausgibt.
- **Großes Sprachmodell:** Sprachmodell, das auf Milliarden einzelner trainierbarer Parameter basiert
- Alle gängigen Großen Sprachmodelle beruhen heute auf der *Transformer-Architektur*

Ablauf der Text-Generierung



Vorhergesagtes Output-Token wird der Eingabesequenz angefügt

→ Vorgang beginnt erneut

„One Token at a Time“

Wie kann Sprache mathematisch verarbeitet werden?

- Problem: Sprache ist einer mathematischen Berechnung grundsätzlich nicht zugänglich
- Daher: Abbildung der semantischen Bedeutung von Sprache im Vektorraum („*vector semantics*“)
- Bsp.: GPT-3 operiert auf Basis von 12.288-dimensionalen Vektoren
- Umsetzung:

Eingabeschicht („ <i>Embedding-Layer</i> “)	→	Vektorisierung der Eingabe
Versteckte Schichten („ <i>Hidden-Layer</i> “)	→	(kontextuelle) Verarbeitung der Vektoren
Ausgabeschicht („ <i>Unembedding-Layer</i> “)	→	Wahrscheinlichkeitsverteilung über Modell-Vokabular

Was sind Token?

- In LLMs werden Texte nicht auf Wortebene verarbeitet, sondern der Text wird segmentiert
- Token können sein: Wörter/Wortteile/Satzzeichen
- Token sind die elementaren Verarbeitungseinheiten eines LLMs
- 1 Token $\approx$ 4 Zeichen bzw. $\frac{3}{4}$ eines Wortes
- Text mit 1.500 Wörtern $\approx$ 2.000 Token
- Bei Zugriff auf LLMs über die API wird nach Token-Anzahl abgerechnet → [Beispiel](#)

Transformer-Architektur - „Attention is all you need“

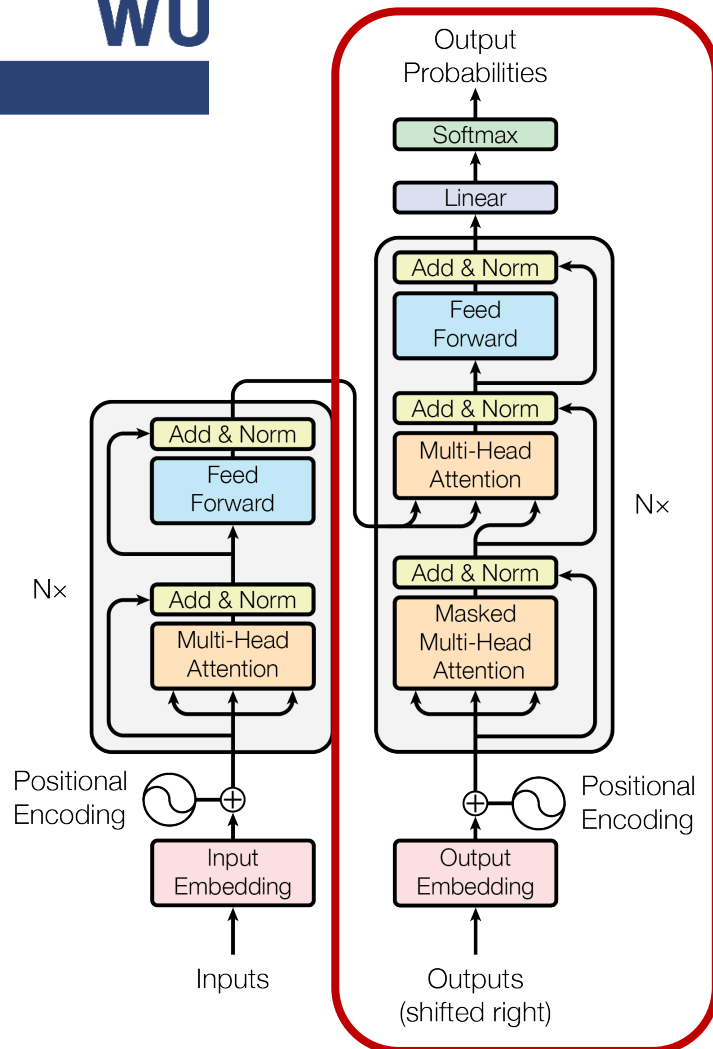


Figure 1: The Transformer - model architecture.

Die ursprüngliche Transformer-Architektur war ein Encoder-Decoder-System zur Übersetzung natürlicher Sprache

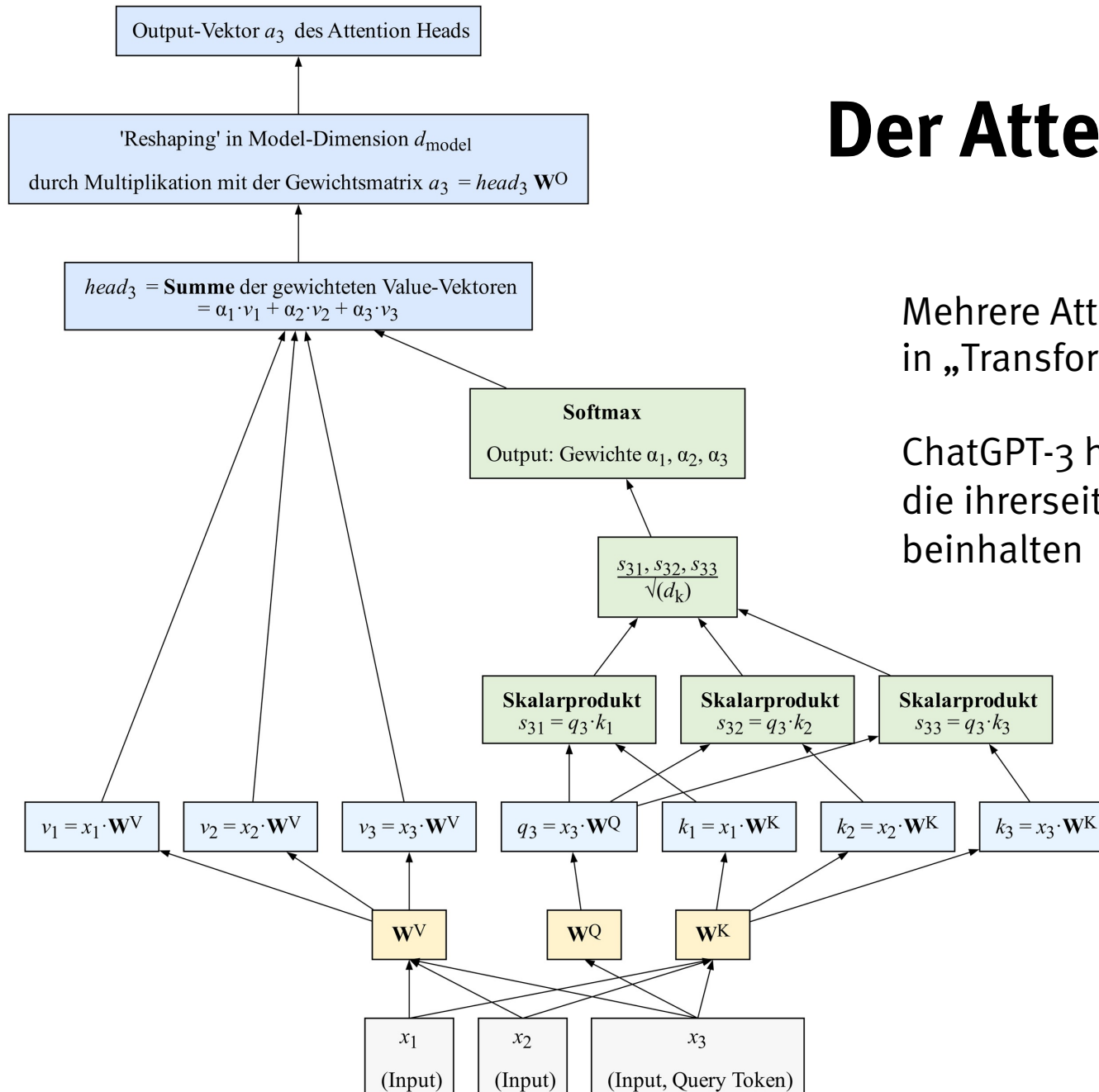
Encoder: Verarbeitung der Eingabesequenz und Umwandlung in Vektorrepräsentationen

Decoder: Schrittweise Generierung der Ausgabesequenz auf Grundlage der Encoder-Repräsentationen und der bereits erzeugten Tokens

Heutige LLMs sind Decoder-Only-Modelle!

Quelle: Vaswani et al., „Attention is all you need“, NIPS 2017

Der Attention-Head



Mehrere Attention-Heads sind eingebettet in „Transformer-Blocks“

ChatGPT-3 hat z.B. 96 Transformer-Blocks, die ihrerseits je 96 Attention-Heads beinhalten

Sampling – Softmax-Funktion

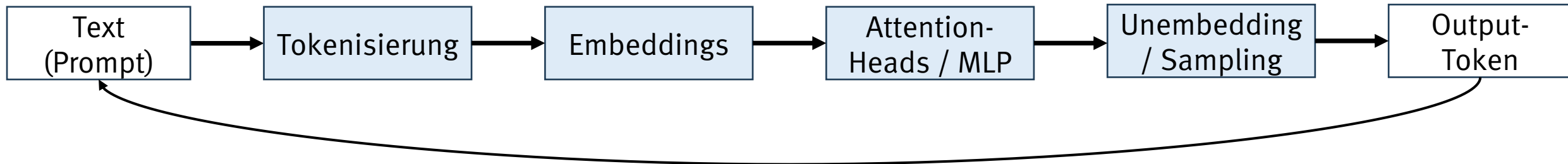
Die Softmax-Funktion „übersetzt“ die Einzelnen Werte des Output-Vektors in eine Wahrscheinlichkeitsverteilung über das Modellvokabular

$$P(v_k) = \frac{e^{l_k}}{\sum_i e^{l_i}}$$

Zur Vertiefung:

Mit dieser Formel wird die Wahrscheinlichkeit berechnet, mit der das Token v_k als Fortsetzung der Eingabesequenz in Betracht kommt. l_k repräsentiert den Logit-Wert, der für das Token k ausgegeben wird. Um daraus einen Wahrscheinlichkeitswert zwischen 0 und 1 berechnen zu können, muss folgender Bruch konstruiert werden $\frac{e^{l_k}}{\sum_i e^{l_i}}$. Die Größe $\sum_i e^{l_i}$ stellt dabei die Summe aller exponentiellen Scores für jeweils alle möglichen Token dar. Der Zähler e^{l_k} verwandelt den Score (der auch negativ sein kann) für das Token k in einen positiven Wert.

Ablauf der Text-Generierung



Vorhergesagtes Output-Token wird der Eingabesequenz angefügt

→ Vorgang beginnt erneut

„One Token at a Time“

Open-Source-Modelle vs. Closed-Source-Modelle

	Open Source:	Closed Source:
Modellparameter	Öffentlich einsehbar	Nicht öffentlich einsehbar
Zugriff	API oder lokales Hosting, z.B. über Ollama	Nur API
Performanz	i.d.R. weniger stark	Stärkere Performance, da regelmäßig wesentlich mehr Parameter
Fine Tuning	Lokal möglich	i.d.R. nicht möglich, teilweise über Anbieter selbst
Beispiele	META LLaMa-Modelle, Google Gemma, Deepseek- Modelle, OpenAI GPT-OSS	OpenAI's ChatGPT-Reihe, Claude-Modelle,